TP Qualité Logicielle — Métriques OO avec CKJM

Projet **Gestion_Bib V2** · outil CKJM 1.8 (résultats identiques avec CKJM 1.9) · JDK 1.8

Partie 1 — Préparation de l'environnement

```
javac -encoding UTF-8 -source 1.8 -target 1.8 -d bin src\library\app\*.java
src\library\model\*.java src\library\service\*.java
java -cp "bin;ckjm-1.8.jar" gr.spinellis.ckjm.MetricsFilter bin\library\app\LibraryApp.class
bin\library\model\Book.class ^
     bin\library\model\DigitalBook.class bin\library\model\Lendable.class
bin\library\service\Customer.class ^
     bin\library\service\Library.class > metrics.txt
```

> **Remarque :** la commande de l'énoncé `java -cp bin;ckjm-1.8.jar -jar ckjm-1.8.jar …` ignore l'option `-cp` (avec `-jar`, Java n'utilise que le classpath du jar). CKJM ne trouve alors pas `Book` (*ClassNotFoundException*) et affiche à tort **DIT = 0** pour `DigitalBook`. En lançant la classe principale `MetricsFilter` avec `-cp`, on obtient la valeur correcte **DIT = 2** (Object → Book → DigitalBook). Le dossier `bin` doit aussi exister avant `javac -d bin`.

1. Synthèse des métriques obtenues (version initiale)

Sortie CKJM : `WMC DIT NOC CBO RFC LCOM Ca NPM`. CKJM 1.8 n'affiche pas Ce ; comme CKJM ignore les classes du JDK, **Ce = CBO** ici (dépendances sortantes vers les classes du projet). I = Ce / (Ca + Ce).

Classe	WMC	DIT	NOC	CBO	RFC	LCOM	Ca	Ce	NPM	I
library.model.Book	7	1	1	1	13	5	3	1	7	0,25
library.model.DigitalBook	2	2	0	1	3	0	1	1	2	0,50
library.model.Lendable	2	1	0	0	2	1	4	0	2	0,00
library.service.Library	5	1	0	2	18	0	1	2	5	0,67
library.service.Customer	3	1	0	1	10	0	0	1	3	1,00
library.app.LibraryApp	2	1	0	4	10	1	0	4	2	1,00

Exercice 1.1 — Signification des métriques CK

Métrique	Nom complet	Ce qu'elle mesure
WMC	Weighted Methods per Class	Complexité interne : nombre de méthodes (poids 1 dans CKJM). Élevé → classe lourde à tester et maintenir.
DIT	Depth of Inheritance Tree	Profondeur d'héritage depuis Object. Élevé → comportement hérité plus difficile à prévoir.
NOC	Number of Children	Nombre de sous-classes directes. Élevé → forte réutilisation, mais une modification impacte beaucoup d'enfants.
CBO	Coupling Between Objects	Nombre d'autres classes auxquelles la classe est couplée. Élevé → fragilité, faible réutilisabilité.
RFC	Response For a Class	Méthodes propres + méthodes appelées (pouvant être exécutées en réponse à un message). Élevé → difficile à tester.
LCOM	Lack of Cohesion of Methods	Paires de méthodes ne partageant aucun attribut moins celles qui en partagent. Élevé → classe à plusieurs responsabilités.
Ca	Afferent Coupling	Classes qui dépendent de celle-ci (entrantes) → responsabilité / stabilité.

Ce	Efferent Coupling	Classes dont celle-ci dépend (sortantes) → instabilité.

Exercice 1.2 — Classes à valeurs élevées

- **a. WMC élevé :** `Book` (7), puis `Library` (5).
- **b. CBO élevé :** `LibraryApp` (4 : Library, Book, DigitalBook, Lendable), puis `Library` (2 : Lendable *et* Book, à cause du `instanceof Book`).
- **c. RFC élevé :** `Library` (18 : ArrayList, Iterator, PrintStream, StringBuilder, getters de Book…), puis `Book` (13).
- **d. LCOM élevé :** `Book` (5) : `complexMethodExample` n'utilise aucun attribut, `getTitle`/`getAuthor` et `borrow`/`returnItem` manipulent des attributs disjoints.

2. Interprétation critique (Exercice 2)

1. **Classe la plus complexe : `Book`.** Elle cumule le WMC (7) et le LCOM (5) les plus élevés, un RFC de 13, et c'est la classe concrète la plus utilisée (Ca = 3, NOC = 1) : toute modification se propage à DigitalBook, Library et LibraryApp. Elle contient en plus `complexMethodExample` (complexité cyclomatique 4) sans rapport avec un livre, et fait de l'affichage console. `Library` a le RFC maximal (18) : c'est la classe la plus difficile à tester isolément.
2. **Classes les plus cohésives : `Library` et `Customer`** (LCOM = 0) : toutes leurs méthodes travaillent sur la même liste (`items` / `checkedOutItems`). `DigitalBook` a aussi LCOM = 0 (une seule méthode utilisant son attribut).
3. **Corrélation WMC / RFC :** oui, positive mais pas proportionnelle (coefficient de Pearson ≈ **0,72** sur les 6 classes). Book (7/13) et Library (5/18) sont en tête des deux. Mais RFC compte aussi les méthodes *appelées* : `LibraryApp` n'a que 2 méthodes mais un RFC de 10, et Library dépasse Book en RFC avec moins de méthodes. Plus de méthodes ⇒ souvent plus de réponses, mais le couplage pèse autant que la taille.
4. **Indicateurs de faible réutilisabilité :** CBO élevé (LibraryApp, Library), et surtout la dépendance de `Library` à la classe concrète `Book` (`instanceof` + cast) : Library ne peut pas gérer d'autre type de `Lendable` (affichage « Unknown item type »). LCOM élevé de Book (responsabilités mélangées), et appels à `System.out` dans les classes métier (RFC gonflé, I/O impossible à réutiliser hors console).
5. **Stabilité des packages** (couplages inter-packages) :

Package	Ca (classes externes dépendantes)	Ce (classes externes utilisées)	I = Ce/(Ca+Ce)	A (abstraction)
library.model	3 (Library, Customer, LibraryApp)	0	0,00	1/3
library.service	1 (LibraryApp)	2 (Lendable, Book)	0,67	0

`library.model` est **le plus stable** (Ca élevé, Ce nul) : c'est normal pour un package métier, mais il doit alors être abstrait (principe SAP). `library.service` est instable et dépend d'une classe concrète du modèle.

3. Propositions de refactoring (Exercice 3.1)

1. **Extraire la méthode de démonstration hors de `Book` (SRP, Extract Class + Extract Method).** `complexMethodExample` est déplacée dans `library.util.NumberClassifier` ; la chaîne if/else-if est extraite dans `classify(int)`, la boucle dans `printSequence(int)`.
2. **Réduire le couplage `Library` → `Book` par l'interface (DIP, polymorphisme).** `Lendable` expose `isBorrowed()` et `getDescription()` ; `Library` n'utilise plus `instanceof`/cast et ne dépend plus que de `Lendable`. Corrige aussi le bug de `getAvailableItems()` qui ignorait l'état emprunté.
3. **Séparer logique métier et affichage.** `borrow()`/`returnItem()` renvoient un `boolean` au lieu d'imprimer un message ; `listAllItems()` renvoie une liste de descriptions et c'est `LibraryApp` qui affiche.
4. **Encapsulation :** attributs `final`, initialisation à la déclaration, ajout de `Customer.getName()` (l'attribut `name` était inutilisé).

Exercice 3.2 — Effet des refactorings sur les métriques

- **Extraction de méthode → ↓ WMC** (pondéré par la complexité) : une méthode longue à plusieurs branches devient des méthodes courtes, chacune de faible complexité cyclomatique ; extraire vers une autre classe diminue le WMC de la classe d'origine.
- **Encapsulation d'attributs → ↓ CBO** : les clients passent par des méthodes de l'objet au lieu de manipuler sa structure ou d'autres objets qu'il contient (loi de Déméter) ; moins de classes connues ⇒ moins de couplage.
- **Fusion ou découpage de classes → ↓ LCOM** : découper une classe qui regroupe des méthodes travaillant sur des attributs disjoints produit des classes cohésives ; fusionner des classes trop éclatées qui partagent les mêmes données réduit aussi le manque de cohésion.
- **Introduction d'interfaces → ↑ abstraction (A) et ↓ instabilité (I)** : les clients dépendent d'une abstraction stable plutôt que de classes concrètes ; le package qui la contient gagne en A, et les packages clients ont moins de dépendances concrètes sortantes (Ce ↓ ⇒ I ↓).

4. Nouvelle version du code (extraits) — dossier `gestion_bib_refactored/`

```
public interface Lendable {                        // library.model
    boolean borrow();  boolean returnItem();
    boolean isBorrowed();  String getDescription();
}

public List<Lendable> getAvailableItems() {         // library.service.Library
    List<Lendable> available = new ArrayList<>();
    for (Lendable item : items)
        if (!item.isBorrowed()) available.add(item);  // bug corrigé
    return available;
}
public List<String> listAllItems() {                 // plus d'instanceof ni de cast
    List<String> descriptions = new ArrayList<>();
    for (Lendable item : items) descriptions.add(item.getDescription());
    return descriptions;
}

public static String classify(int i) {               // library.util.NumberClassifier
    if (i % 2 == 0) return "is even";
    if (i % 3 == 0) return "divisible by 3";
    return "something else";
}
```

Le programme produit exactement la même sortie qu'avant (liste des 3 livres puis la séquence 0 à 9).

5. Comparaison avant / après refactoring

Classe	WMC	CBO	RFC	LCOM	Ca	NPM
Book	7 → 7	1 → 1	13 → 11	5 → 5	3 → 2	7 → 7
DigitalBook	2 → 2	1 → 1	3 → 3	0 → 0	1 → 1	2 → 2
Lendable	2 → 4	0 → 0	2 → 4	1 → 6	4 → 4	2 → 4
Library	5 → 5	2 → 1	18 → 14	0 → 0	1 → 1	5 → 5
Customer	3 → 4	1 → 1	10 → 11	0 → 0	0 → 0	3 → 4
LibraryApp	2 → 2	4 → 5	10 → 13	1 → 1	0 → 0	2 → 2
NumberClassifier (nouvelle)	3	0	9	3	1	2

Indicateur	Avant	Après
Complexité cyclomatique totale de Book	12	9
Dépendances de `Library` vers des classes concrètes	1 (Book)	0
Instabilité du package `service` (Ce/(Ca+Ce))	0,67	0,50
Abstraction du package `model`	1/3	1/3 (interface enrichie)

Évolution : les gains portent sur les classes visées : **Library** (CBO 2→1, RFC 18→14) n'est plus liée à Book, et **Book** perd une dépendance entrante (Ca 3→2), son RFC (13→11) et 3 points de complexité cyclomatique. Le WMC de Book reste 7 car CKJM donne un poids de 1 à chaque méthode : la méthode complexe a été remplacée par un simple `getDescription()`, ce que CKJM ne distingue pas. Le LCOM de Book reste 5 : la classe mélange encore informations bibliographiques (title, author) et état de prêt (borrowed). Les hausses sont attendues : le LCOM de `Lendable` (6) est un artefact (une interface n'a pas d'attributs, donc aucune paire de méthodes n'en partage) ; `LibraryApp` (CBO 5, RFC 13) est le point d'assemblage de l'application, où le couplage est normal (I = 1).

6. Conclusion

CKJM a permis de repérer objectivement les points faibles : `Book` peu cohésive et trop sollicitée, `Library` couplée à une classe concrète avec un RFC élevé. Le refactoring (extraction de classe/méthode, dépendance à l'interface, séparation métier/affichage) rend `Library` extensible à tout nouveau type de `Lendable` sans modification (principe ouvert/fermé), rend les classes du modèle testables sans console, et corrige un bug fonctionnel. Les métriques montrent aussi leurs limites : leur somme ne baisse pas forcément (nouvelle classe, interface plus riche), et CKJM ne mesure ni la complexité cyclomatique ni le sens des dépendances. Il faut donc les interpréter classe par classe et en lien avec la conception. Prochaine étape possible : séparer l'état de prêt de `Book` (classe `Loan` ou classe abstraite `AbstractLendable`) pour réduire son LCOM.